

How an AI Agent Orders Food

A plain-language walkthrough of what actually happens, system by system, when someone asks an AI assistant to check a dish and order it — written for readers outside engineering.

"Check if Paneer Butter Masala is available at Spice Route, and if it is, order me one."

Contents

- 1 Who This Handbook Is For
- 2 The Cast of Characters
- 3 The Big Picture — How Everything Connects
- 4 Walking Through One Order, Start to Finish
 - 4.1 Step 1–3: Finding and checking the dish
 - 4.2 Step 4–7: Placing the order
 - 4.3 Step 8–10: Getting the food to the table
- 5 The Six Tools the AI Agent Can Use
- 6 The Life of an Order (State by State)
- 7 When Things Go Wrong
- 8 Glossary of Terms
- 9 Frequently Asked Questions

1 Who This Handbook Is For

This document explains, in plain language, exactly what happens behind the scenes when a person asks an AI assistant like Claude to check on a food item and order it through the Bharat MCP system. No coding background is assumed. Wherever a technical idea shows up, it's paired with an everyday comparison first.

THINK OF IT THIS WAY

Imagine a large hotel where a guest never talks to the kitchen directly. Instead, the guest tells the **concierge** what they want, the concierge calls the kitchen, checks what's available, places the order, and comes back to tell the guest when it will arrive. The guest never sees the kitchen, the pantry, or the phone call — they just experience the request and the result. That concierge is what the AI agent is doing here, and this handbook is a tour of everything the concierge does that the guest never sees.

Every technical term used later in this handbook (MCP, tool call, API, relay, and so on) is also collected in the **Glossary** in Section 8, so you can jump there any time a word doesn't land.

2 The Cast of Characters

Before following one order end to end, it helps to know who's involved. Five things are doing work in this story — think of them as five characters in the same play.

CHARACTER	WHAT IT ACTUALLY IS	WHAT IT DOES IN THIS STORY
The AI Agent	Claude, ChatGPT, or Gemini — the assistant the person is chatting with	Talks to the person in plain language, decides what needs to be checked or done, and asks Bharat MCP for it
Bharat MCP Server	A piece of software Reward360 built and runs	The only door the AI agent can knock on. It understands requests like "check availability" or "place an order" and knows how to fulfil them
The Database	A structured, searchable filing cabinet (Postgres)	Holds every restaurant's menu, prices, hours, and every order ever placed
The Restaurant	A real restaurant with a phone and a WhatsApp number	Receives the order as a message, and a person there accepts or rejects it
The Payment Gateway	Razorpay or Cashfree — a licensed payment company	Collects the customer's money via UPI and eventually pays the restaurant

ONE IMPORTANT RULE

The AI agent never touches the database directly, never messages the restaurant directly, and never touches money directly. It only ever does one thing: it asks the Bharat MCP server a question or gives it an instruction, in a fixed, pre-agreed format. Everything else happens on the other side of that door. This rule is what keeps the whole system safe, testable, and consistent no matter which AI platform is asking.

3 The Big Picture — How Everything Connects

Here is the full map. The person only ever sees the top box. Everything below it is what this handbook is about.

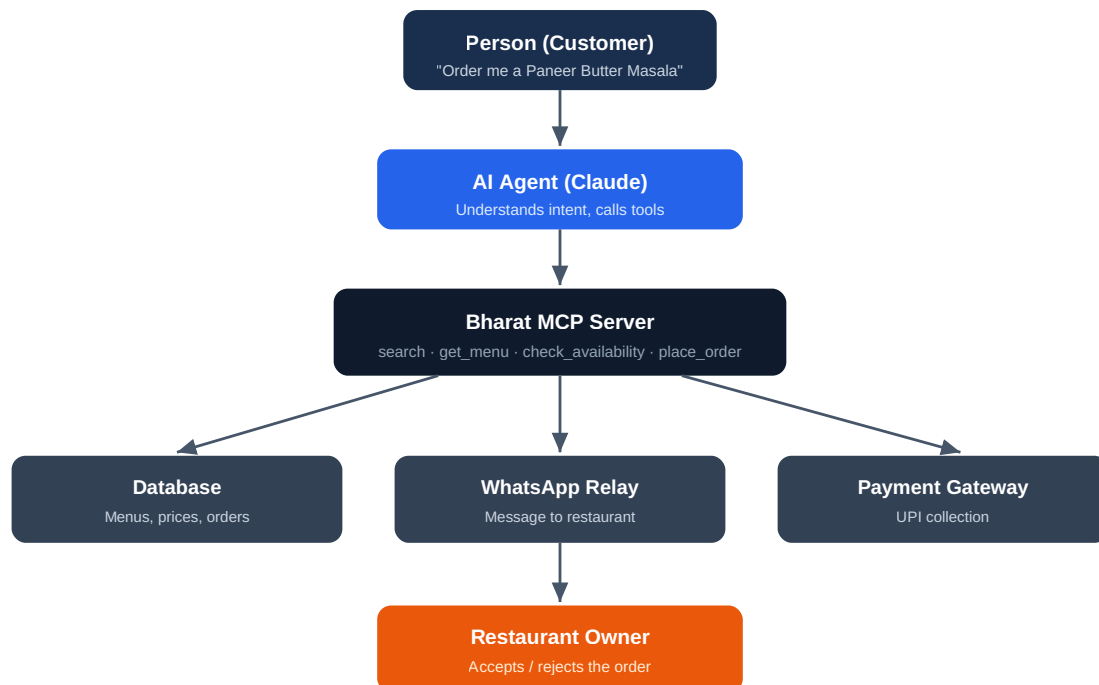


Fig 1. The customer only ever sees the top two boxes. Everything below the AI Agent is invisible to them, but it's where all the real work happens.

Why it's built this way

Notice that the AI agent talks to exactly **one** thing — the Bharat MCP Server — and never to the database, WhatsApp, or the payment gateway directly. This is deliberate. It means Claude, ChatGPT, and Gemini can all use the exact same system without Reward360 building three different integrations, and it means the restaurant's data and the customer's payment stay behind a single, well-guarded front door rather than being scattered across many connections.

4 Walking Through One Order, Start to Finish

Now the full journey, using exactly the example from the cover: a person asks Claude to check whether Paneer Butter Masala is available at a restaurant, and then order it. Every step below is something that really happens on a server somewhere, in well under two seconds combined for the first few steps.

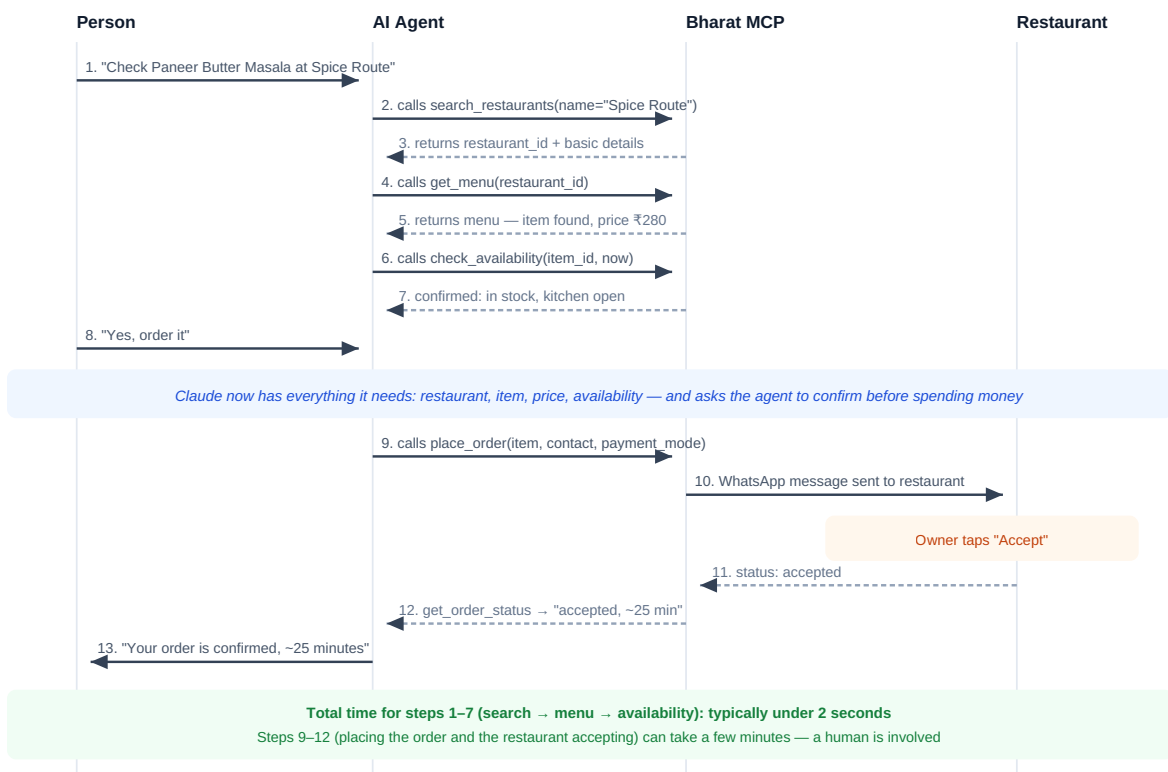


Fig 2. Solid arrows are requests going out; dashed arrows are answers coming back. Every solid/dashed pair is one "tool call."

4.1 Finding and checking the dish (Steps 1–7)

1

The person asks a plain-language question

No special commands, no app to open. Just: "Check if Paneer Butter Masala is available at Spice Route."

2

Claude translates this into a structured request

Claude doesn't "guess" the restaurant's data — it can't see it. So it calls a specific tool on Bharat MCP: `search_restaurants(query="Spice Route")`. This is exactly like the concierge picking up the phone and dialling the kitchen instead of imagining what's in the fridge.

WHAT IS A "TOOL CALL," REALLY?

A tool call is a small, fixed-format request — almost like filling out a very short form. The AI agent doesn't write free-flowing sentences to the server; it fills in specific named fields (like `restaurant_id` or `item_ids`) that the Bharat MCP server has told it, in advance, that it understands. This fixed format is what "MCP" (Model Context Protocol) standardises, so that Claude, ChatGPT, and Gemini can all fill out the exact same form the exact same way.

3 Bharat MCP looks up the restaurant in its database

The server checks Postgres (the database) for a restaurant named "Spice Route" that is currently marked `active`, and returns its internal ID and basic details. This lookup is required to happen in under 1.5 seconds, per the system's own requirements.

4 Claude asks for the full menu

Using the restaurant's ID from step 3, Claude calls `get_menu(restaurant_id)`. This returns every category and item, with prices — and importantly, items that are sold out are still shown, just marked unavailable, rather than silently disappearing.

5–6 Claude finds the dish and double-checks it's actually available right now

A menu listing isn't the same as "in stock right now." So a second, real-time check runs: `check_availability()` confirms the kitchen is open at this hour and the item hasn't just sold out in the last few minutes.

7 Claude now has a real, confirmed answer

Not "I think it's ₹280," but "it is ₹280, it's in stock, right now, at this specific restaurant." Claude reports this back to the person and — because money is about to be spent — asks for a clear "yes" before going further.

4.2 Placing the order (Steps 8–11)

8 The person confirms

"Yes, order it." This is the moment the AI agent is authorised to actually spend money and create a real order — everything before this was just looking, not committing.

9 Claude calls `place_order()`

This one call carries everything needed: which restaurant, which item, how the customer can be reached, and how they want to pay (UPI or cash on delivery). Bharat MCP creates a real row in the `orders` table in the database — this is the order's official birth.

A SAFEGUARD WORTH KNOWING ABOUT

Every order carries something called an **idempotency key** — a unique tag for that specific order attempt. If, for any reason, the same "place order" instruction gets sent twice (a network hiccup, the agent retrying automatically), the server recognises the duplicate tag and returns the *same* order instead of accidentally creating two orders and charging twice. This is a deliberately engineered protection, not an accident of how it happens to work.

10 The order is relayed to the restaurant — by WhatsApp

Most of these restaurants have no point-of-sale system or order screen. So instead of a digital ticket appearing in a kitchen display, a WhatsApp message lands on the restaurant's registered phone, required to arrive within 30 seconds of the order being created.

11 A real person at the restaurant accepts or rejects it

This is the one step in the entire journey that isn't fully automatic — it depends on a human replying. If nobody replies within a set time window, the order is automatically flagged for a Bharat MCP team member to follow up on manually, rather than leaving the customer waiting indefinitely.

4.3 Getting the answer back to the person (Step 12–13)

12 The status flows back up the same chain it came down

The restaurant's reply updates the order's status in the database. Claude checks this using `get_order_status(order_id)` and translates it back into a plain sentence for the person: "Your order is confirmed, ready in about 25 minutes." The person never sees any of the tool calls, the database row, or the WhatsApp message — only this final sentence.

5 The Six Tools the AI Agent Can Use

Everything in Section 4 is really just these six tools, called in different combinations. This is the complete list — the AI agent cannot do anything with Bharat MCP beyond what's in this table.

TOOL NAME	IN PLAIN WORDS	WHEN CLAUDE WOULD USE IT
<code>search_restaurants</code>	"Find restaurants matching this name, cuisine, or area"	The very first step of almost any request
<code>get_menu</code>	"Show me everything this restaurant sells, and the prices"	Once a restaurant is identified
<code>check_availability</code>	"Is this specific item actually in stock right now, and is the kitchen open?"	Right before confirming anything to the customer
<code>place_order</code>	"Create this order for real, and notify the restaurant"	Only after the customer has clearly said yes
<code>get_order_status</code>	"What's happened to this order since I placed it?"	To follow up, or when the customer asks "where's my food"
<code>book_table</code>	"Reserve a table for a certain number of people at a certain time"	Only for restaurants that have opted into table bookings

WORTH NOTICING

Nowhere in this list is a tool like "charge this card" or "delete this restaurant." The AI agent's permissions are intentionally narrow — it can read menus and place orders, and nothing else. This is enforced by the server, not by trusting the AI agent to behave — even if an AI agent "wanted" to do something outside this list, there is no tool for it to call.

6 The Life of an Order (State by State)

Every order moves through a small, fixed set of stages. An order is always in exactly one of these at any moment — this is what `get_order_status` is actually reporting on.

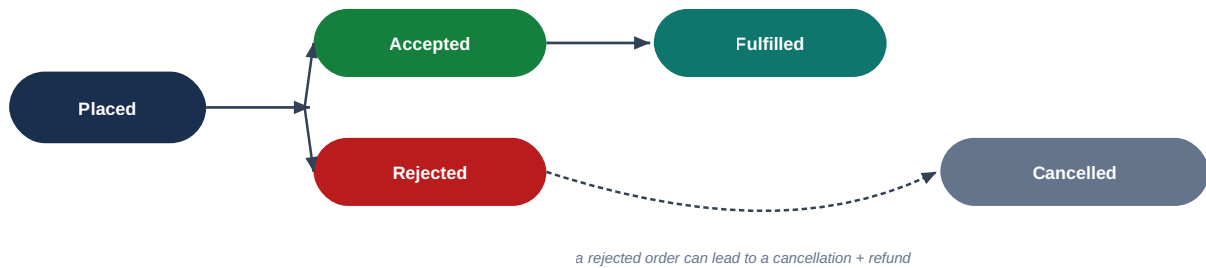


Fig 3. Every order lives in exactly one of these five states at any given time.

STATE	WHAT IT MEANS
Placed	The order exists in the database and the WhatsApp message has been sent to the restaurant. Nobody has responded yet.
Accepted	The restaurant owner has confirmed they will prepare it. The customer can be told a rough ready time.
Rejected	The restaurant can't fulfil it (sold out, closing, too busy). The customer needs to be told immediately, and if they already paid, this is where a refund conversation begins.
Fulfilled	The order has been completed — picked up, served, or delivered, depending on the pilot's chosen model.
Cancelled	The order was called off, either by the customer, the restaurant, or automatically after a no-response timeout.

7 When Things Go Wrong

Not every order goes as smoothly as the example in Section 4. Here's what happens in the situations that actually occur regularly at this scale, and how the system is designed to handle each one without leaving the customer confused.

THE DISH IS ACTUALLY SOLD OUT

The `get_menu` or `check_availability` call returns "unavailable" instead of confirming stock. Claude is expected to tell the person clearly rather than attempting the order anyway — this is exactly why the availability check (step 6 in Section 4) happens *before* the order is placed, not after.

THE RESTAURANT NEVER RESPONDS ON WHATSAPP

After a configurable waiting period (for example, 10 minutes), the order is automatically flagged inside the internal admin tool for a human on the Bharat MCP team to step in — call the restaurant directly, or cancel and refund the order. The AI agent is told the order has "timed out" and can pass that along honestly rather than leaving the person guessing.

THE PAYMENT FAILS PARTWAY THROUGH

If the payment gateway's confirmation message doesn't arrive as expected, the system automatically retries checking at least three times before giving up and raising an alert — it does not silently assume the payment worked.

THE AI PLATFORM ITSELF HAS AN ISSUE, OR THE SERVER IS DOWN

Bharat MCP is monitored continuously; if the server that all six tools depend on stops responding, an alert reaches the engineering team within five minutes. Until it's back up, the AI agent will get an error instead of an answer, and should tell the customer the service is temporarily unavailable rather than guessing at a response.

The one thread running through every failure case above: **the AI agent is never allowed to guess**. If it doesn't get a clear, real answer from Bharat MCP, it says so honestly rather than inventing one — this is a deliberate design choice, not a limitation that slipped through.

8 Glossary of Terms

MCP (Model Context Protocol)

A shared, standard format for how an AI agent asks an outside system to look something up or do something. It's the reason Claude, ChatGPT, and Gemini can all use the same Bharat MCP server without three separate custom integrations.

Tool / Tool Call

One specific, named action the AI agent is allowed to request — like `get_menu` or `place_order` — filled in with the specific details needed for that request.

API

A general term for "a defined way for one piece of software to ask another piece of software for something." MCP is a particular kind of API designed specifically for AI agents.

Relay

The WhatsApp/SMS message that carries a confirmed order to a restaurant that has no digital order system of its own.

Idempotency Key

A unique tag attached to an order attempt so that if the same request accidentally arrives twice, the system recognises it and doesn't create — or charge for — a duplicate order.

Tenant

One restaurant's own slice of data (its menu, hours, orders) living inside the single shared Bharat MCP system, alongside all 499 others.

Database (Postgres)

The structured filing system holding every restaurant's menu, prices, and order history — think of it as a very fast, very organised spreadsheet that many programs can read and write to at once.

Smoke Test

A quick, automatic trial run (a fake `get_menu` and a test `place_order`) performed on a restaurant's data before it's allowed to go live, to catch obvious mistakes early.

Webhook

A message a payment provider sends back automatically to confirm "this payment succeeded" or "this payment failed," without anyone needing to check manually.

9 Frequently Asked Questions

Q: Does the AI agent ever see the restaurant's actual database?

No. It only ever sees what a tool call returns — a clean, filtered answer to a specific question. It has no broader access to the database itself.

Q: Can the AI agent place an order without the person's confirmation?

The system is designed around the person clearly saying "yes" before `place_order` is called — the earlier steps (search, menu, availability) are all read-only and don't commit to anything.

Q: What if two different people order the same last remaining dish at the same moment?

The availability check happens as close to order time as possible to minimise this, but because a real person at the restaurant still has to accept the order, the restaurant is the final check — this is one reason the accept/reject step exists rather than treating an order as final the moment it's placed.

Q: Why does the restaurant get a WhatsApp message instead of the order just appearing on a screen?

Because most small, independent restaurants in this pilot don't have a point-of-sale or order-management system at all. WhatsApp is something every restaurant owner already has and checks — it's a deliberate low-tech bridge for this specific pilot, not a long-term design choice.

Q: Does this handbook cover how restaurants get onboarded, or how payments are legally structured?

No — this handbook covers only the technical flow of a single order. Restaurant onboarding, legal/compliance requirements, and payment-settlement structure are covered in the companion documents: the Infrastructure Plan, the Functional Requirements Document, and the Non-Technical Requirements document.